FASE 1 · PROYECTO 01

rawio

Hablando directamente con el kernel:
syscalls, file descriptors y sistema de ficheros

Versión 1.0 · Roadmap Sistemas Linux

Resumen

Vas a reescribir versiones mínimas de `cat`, `cp`, `tail` y `ls` usando únicamente llamadas al sistema, sin `stdio`. El objetivo no es la utilidad en sí, sino entender qué ocurre entre tu programa y el kernel cada vez que lees o escribes un byte: escrituras parciales, interrupciones, buffers, metadatos e inodos.

CAPÍTULO I

Introducción

Cuando llamas a `printf`, la libc acumula los datos en un buffer en *user space* y, en algún momento, ejecuta la syscall `write(2)`. Esa syscall provoca un cambio de modo: la CPU pasa de Ring 3 a Ring 0, el kernel valida tus argumentos, copia los datos y vuelve. Todo lo que la libc te oculta (errores, escrituras parciales, el coste del cambio de modo) es lo que vas a manejar tú en este proyecto.

Un *file descriptor* es solo un entero: un índice en la tabla de ficheros abiertos de tu proceso. Detrás hay una *open file description* en el kernel (con su offset y sus flags) que apunta a un *inodo* (los metadatos del fichero). Distinguir estas tres capas es la clave para entender `dup`, `fork` y los pipes en el proyecto siguiente.

CAPÍTULO II

Instrucciones generales

- Lenguaje C. Compila con `cc -Wall -Wextra -Werror -std=c17 -D_DEFAULT_SOURCE`.
- **Prohibido** `stdio.h` para entrada/salida (`printf`, `fopen`, `fgets`…). Escribe tus propias funciones de salida sobre `write`. Puedes usar `snprintf` solo en el ejercicio ex04 para formatear números.
- Cada ejercicio va en su propio directorio (`ex00/`, `ex01/`…) con su Makefile.
- Los mensajes de error van a `stderr` con el formato `programa: contexto: mensaje`, usando `strerror(errno)`.
- El código de salida debe ser 0 si todo fue bien y distinto de 0 si hubo algún error, como las herramientas reales.
- Sin fugas de memoria ni de file descriptors al terminar.

CAPÍTULO III

Ejercicios

Ejercicio	00: strace_diary
Entrega	`ex00/NOTES.md`
Permitido	`strace`, `ltrace`, `man`

Antes de programar nada, observa. Escribe un «hola mundo» con `printf`, compílalo y ejecuta `strace ./hola`, `strace cat /etc/hostname` y `strace -c ls /usr/bin`. Responde en `NOTES.md`:

1. ¿Cuál es la primera syscall de cualquier programa y quién la hace?
2. ¿Para qué sirven `brk`, `mmap` y `mprotect` antes de que llegue a ejecutarse tu `main`?
3. ¿Por qué ves `openat` y no `open`?
4. Ejecuta tu «hola mundo» con 3 `printf` hacia la terminal y luego redirigido a un fichero (`./hola > out`). ¿Cuántos `write` ves en cada caso? Explica por qué cambia.
5. En la salida de `strace -c ls`, ¿qué syscall domina y por qué?

Ejercicio	01: mcat
Entrega	`ex01/` · binario `mcat`
Permitido	`open`, `close`, `read`, `write`, `strerror`, `exit`

Replica el comportamiento básico de `cat`: concatena los ficheros recibidos como argumento en la salida estándar. Sin argumentos, o con el argumento `-`, lee de la entrada estándar.

- `write` puede escribir **menos** bytes de los pedidos. Tu programa debe completar siempre la escritura.
- Si `read` o `write` fallan con `EINTR`, reintenta.
- Si un fichero no existe, muestra el error, continúa con los siguientes y termina con código 1.
- Usa un buffer fijo; nunca cargues el fichero entero en memoria.

```
$ ./mcat no_existe /etc/hostname
mcat: no_existe: No such file or directory
mi-maquina
$ echo $?
1
```

PISTA

Para provocar escrituras parciales, escribe a un pipe que se lee despacio: `./mcat fichero_grande | (sleep 1; head -c 10)`. ¿Qué señal recibe tu programa cuando `head` termina? Lo verás a fondo en `procs`.

Ejercicio	02: mcp
Entrega	`ex02/` · binario `mcp`, `BENCH.md`
Permitido	lo de ex01 + `fstat`, `stat`, `unlink`, `clock_gettime`

`mcp [-b SIZE] origen destino` copia un fichero regular.

- El destino se crea con los mismos bits de permisos que el origen (los 12 bits bajos de `st_mode`) y se trunca si ya existía.
- Si origen y destino son **el mismo fichero** (aunque se llamen distinto, por ejemplo vía un hard link), rechaza la copia. Pista: un fichero queda identificado por el par `st_dev` + `st_ino`.
- Si la copia falla a mitad, no dejes un destino corrupto: elimínalo.
- `-b SIZE` fija el tamaño del buffer en bytes (por defecto 65536).

En `BENCH.md`, crea un fichero de 200 MiB (`dd if=/dev/urandom of=big bs=1M count=200`) y mide la copia con buffers de 1, 64, 4096, 65536 y 1048576 bytes. Presenta una tabla con el tiempo real y el número de syscalls (`strace -c -f`), y explica la curva. ¿Por qué deja de mejorar a partir de cierto tamaño?

Ejercicio	03: mtail
Entrega	`ex03/` · binario `mtail`
Permitido	lo anterior + `lseek`, `malloc`, `free`

`mtail [-n N] [fichero]` muestra las últimas N líneas (10 por defecto).

- Con un fichero regular, **no puedes leerlo entero**: usa `lseek` para ir al final y lee bloques hacia atrás hasta haber encontrado suficientes saltos de línea. Debe responder al instante con un fichero de 10 GiB.
- Si la entrada no admite `lseek` (stdin conectado a un pipe), `lseek` falla con `ESPIPE`. En ese caso, lee en flujo y guarda solo las últimas N líneas (buffer circular).
- Cuidado con la última línea sin `\n` final y con los ficheros vacíos.

Ejercicio	04: mls
Entrega	`ex04/` · binario `mls`
Permitido	lo anterior + `opendir`, `readdir`, `closedir`, `lstat`, `readlink`, `getpwuid`, `getgrgid`, `time`, `localtime`, `strftime`, `snprintf`

`mls [-a] [-l] [ruta...]` lista directorios. Sin `-a`, omite las entradas que empiezan por punto. La salida va ordenada alfabéticamente por nombre (implementa tu propio ordenamiento).

Con `-l`, el formato debe coincidir con `LC_ALL=C ls -l` (columnas alineadas incluidas):

```
$ ./mls -l /tmp/demo
total 12
drwxr-xr-x 2 bs   bs   4096 Oct  8 10:12 dir
lrwxrwxrwx 1 bs   bs     11 Oct  8 10:12 enlace -> fichero.txt
-rw-r--r-- 1 bs   bs     13 Oct  8 10:11 fichero.txt
-rwsr-xr-x 1 root root 1024 Mar  2  2025 viejo
```

- Tipo de fichero y permisos, incluidos los bits especiales (`s`, `S`, `t`, `T`).
- Para enlaces simbólicos usa `lstat` (no `stat`) y muestra el destino con `readlink`.

- Fechas de más de seis meses (o en el futuro) se muestran con el año en lugar de la hora, como `ls`.
- La línea `total` suma `st_blocks` expresado en bloques de 1 KiB.
- Si el usuario o grupo no existe, muestra el UID/GID numérico.

CAPÍTULO IV

Parte bonus

El bonus solo se evalúa si la parte obligatoria es perfecta.

- `mls -R`: listado recursivo. Ojo con los ciclos (enlaces simbólicos a directorios padre).
- `mcp` con soporte de **ficheros dispersos** (*sparse files*) usando `lseek` con `SEEK_DATA` y `SEEK_HOLE`: una copia de un fichero de 1 GiB con un solo byte escrito debe ocupar lo mismo en disco que el original (compruébalo con `du`).
- `mcp --mmap`: segunda implementación con `mmap(2)`. Añade el resultado a `BENCH.md` y explica las diferencias.
- `mcat` con `splice(2)` o `sendfile(2)` cuando la entrada y la salida lo permitan (copia sin pasar por user space).

CAPÍTULO V

Evaluación

Prueba	Esperado
`NOTES.md` de ex00	Respuestas correctas y razonadas, no copiadas: el evaluado debe saber explicarlas en voz alta.
`./mcat f1 f2 - f3 < f4`	Mismo resultado que `cat`.
`./mcat bigfile | head -c 1`	Termina inmediatamente, sin colgarse.
`./mcp f f` y `./mcp f hardlink_de_f`	Ambos rechazados con un mensaje claro; el fichero queda intacto.
`./mcp f /directorio_sin_permiso/x`	Error claro, código de salida ≠ 0, sin restos.
`BENCH.md`	Tabla coherente y explicación del coste del cambio de modo y de la page cache.
`./mtail -n 3 fichero_10GiB`	Respuesta instantánea; `strace` muestra unas pocas lecturas.
`seq 100000 | ./mtail -n 2`	`99999` y `100000`.
`diff <(LC_ALL=C ls -la /etc) <(./mls -la /etc)`	Sin diferencias.
`valgrind --track-fds=yes` sobre cada binario	Sin fugas de memoria ni de file descriptors.

ATENCIÓN

Durante la corrección, el evaluador elegirá dos syscalls al azar de tu código y te preguntará qué errores pueden devolver y cómo los gestionas. «No pueden fallar» no es una respuesta válida.